

MODEL VALIDATION, OVERFITTING CONTROL AND HYPERPARAMETER TUNING

California Housing Dataset

ABSTRACT

This project focuses on implementing model validation, overfitting detection, cross-validation, and hyperparameter tuning using machine learning algorithms on the California Housing Dataset. The workflow includes data preprocessing, feature scaling, model training, overfitting analysis, cross-validation, and parameter optimization. Python libraries such as Pandas, NumPy, Matplotlib, and Scikit-learn were used for implementation. Different regression models including Linear Regression, Ridge Regression, and Decision Tree Regressor were evaluated using RMSE and R^2 Score. The results showed that hyperparameter tuning and cross-validation improve model generalization and prediction reliability.

1. INTRODUCTION

Machine Learning models often suffer from overfitting, where the model performs well on training data but poorly on unseen data. Model validation techniques are used to improve reliability and ensure better generalization.

Cross-validation and hyperparameter tuning are important techniques used in machine learning to optimize model performance and reduce overfitting. Hyperparameter tuning helps in selecting the best model parameters, while cross-validation provides stable and reliable evaluation results.

In this project, different regression models were implemented using the California Housing Dataset. The project demonstrates important machine learning concepts including feature scaling, model evaluation, overfitting detection, cross-validation, and GridSearchCV optimization.

2. DATA PREPROCESSING

The California Housing Dataset from Scikit-learn was used for this task. The dataset contains housing-related features such as:

- Median Income
- House Age
- Average Rooms
- Population
- Latitude and Longitude

The following preprocessing steps were performed:

- Loading the dataset
- Separating features and target variable

- Feature scaling using StandardScaler
- Splitting the dataset into training and testing sets

The dataset was divided using an 80:20 train-test ratio.

3. MODEL TRAINING AND OVERFITTING ANALYSIS

The following regression models were implemented:

- Linear Regression
- Ridge Regression
- Decision Tree Regressor

The Decision Tree model was used to detect overfitting by comparing training RMSE and testing RMSE values.

Observations:

- Training RMSE was significantly lower
- Testing RMSE was comparatively higher

This indicated that the model was overfitting the training data and failed to generalize properly on unseen data.

8. Detect Overfitting Using Decision Tree

```
tree = DecisionTreeRegressor(random_state=42)

tree.fit(X_train, y_train)

train_pred = tree.predict(X_train)
test_pred = tree.predict(X_test)

train_rmse = np.sqrt(mean_squared_error(y_train, train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test, test_pred))

print("Training RMSE:", train_rmse)
print("Testing RMSE:", test_rmse)
```

[8]

```
... Training RMSE: 3.0302402915898627e-16
Testing RMSE: 0.7059501046231196
```

Figure 1: Training and Testing RMSE Output

4. CROSS-VALIDATION AND HYPERPARAMETER TUNING

5-fold cross-validation was performed using `cross\_val\_score()` to obtain reliable model evaluation results.

Advantages of Cross-Validation:

- Reduces dependency on a single train-test split
- Improves reliability of evaluation

- Provides stable performance estimates

Hyperparameter tuning was performed using `GridSearchCV()` by optimizing:

- `max\_depth`
- `min\_samples\_split`

The optimized model reduced overfitting and improved prediction performance.

9. Cross Validation

```
cv_scores = cross_val_score(
    tree,
    X_scaled,
    y,
    scoring='neg_root_mean_squared_error',
    cv=5
)

cv_rmse = -cv_scores.mean()

print("Cross Validation RMSE:", cv_rmse)
```

[9] ... Cross Validation RMSE: 0.9029281160721967

10. Hyperparameter Tuning Using GridSearchCV

```
param_grid = {
    'max_depth': [3, 5, 7, 10],
    'min_samples_split': [2, 5, 10]
}

grid = GridSearchCV(
    DecisionTreeRegressor(random_state=42),
    param_grid,
    scoring='neg_root_mean_squared_error',
    cv=5
)

grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
```

[10] ... Best Parameters: {'max_depth': 10, 'min_samples_split': 10}

Figure 2: Cross-Validation and GridSearchCV Output

5. EVALUATION METRICS

The models were evaluated using:

- Root Mean Squared Error (RMSE)
- R² Score

Model	RMSE	R ² Score
Linear Regression	0.745581	0.575788
Ridge Regression	0.745554	0.575819
Optimized Decision Tree	0.645430	0.682099

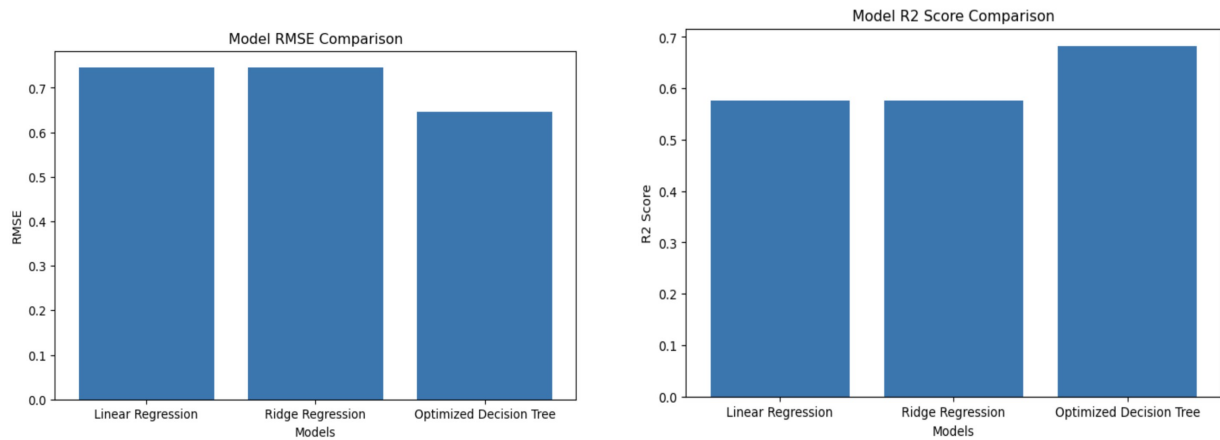


Figure 3: Model Comparison Graph

Interpretation:

- Lower RMSE indicates better prediction accuracy.
- Higher R^2 Score indicates better model performance.
- Hyperparameter tuning improved generalization capability.

6. CONCLUSION

This project successfully demonstrated model validation, overfitting detection, cross-validation, and hyperparameter tuning using the California Housing Dataset.

The workflow included:

- Data preprocessing
- Feature scaling
- Model training
- Overfitting analysis
- Cross-validation
- Hyperparameter tuning
- Model evaluation
- Visualization

The results showed that cross-validation and GridSearchCV significantly improved model reliability and reduced overfitting. The optimized Decision Tree model achieved better generalization performance on unseen data and provided more accurate predictions.